Moshitha K 192421205

SECTION A – DEBUGGING QUESTIONS

1. Diamond Problem in Multipath Hybrid Inheritance

Topic: Hybrid inheritance and virtual base classes

1) The compiler reports an ambiguous member access for ta.idNumber because TeachingAssistant contains two Person subobjects.

2) Non-virtual layout: TeachingAssistant contains a Student branch with its own Person(idNumber=101), and an Employee branch with its own Person(idNumber=202). Therefore ta.idNumber does not identify a unique field.

3) Corrected virtual inheritance:

```
#include <iostream>
class Person {
public:
   int idNumber;
   Person(int id=0) : idNumber(id) {
      std::cout << "Person initialized: " << idNumber << '\n';
   }
};
class Student : virtual public Person {
public:
   Student(int id) : Person(id) {}
};
class Employee : virtual public Person {
public:
   Employee(int id) : Person(id) {}
};
class TeachingAssistant : public Student, public Employee {
public:
   TeachingAssistant(int id)
      : Person(id), Student(id), Employee(id) {}
};
int main() {
   TeachingAssistant ta(101);
   std::cout << "TA ID: " << ta.idNumber << '\n';
}
```

4) With virtual inheritance there is one shared Person subobject. The most-derived class, TeachingAssistant, is responsible for initializing that virtual base. Person is initialized before Student and Employee.

Output:

```
Person initialized: 101
TA ID: 101
```

2. Missing Base Constructor in Derived Initializer List

Topic: Derived constructors and execution order

1) Device has only Device(std::string); it has no default constructor. Since Sensor does not explicitly initialize Device, the compiler tries to call Device() before entering the Sensor constructor body. That call does not exist, so compilation fails.

2) Construction order: virtual base classes first; then non-virtual base classes in declaration order; then data members in declaration order; finally the derived constructor body. Here Device is constructed before Sensor's member initialization/body.

3–4) Correct code and output:

```
#include <iostream>
#include <string>
class Device {
protected:
   std::string macAddress;
public:
   Device(std::string mac) : macAddress(mac) {
      std::cout << "Device configured with MAC: " << macAddress << '\n';
   }
};
class Sensor : public Device {
   int samplingRate;
public:
   Sensor(std::string mac, int rate)
      : Device(mac), samplingRate(rate) {}
   void display() const {
      std::cout << "MAC: " << macAddress
            << ", Rate: " << samplingRate << " Hz\n";
   }
};
int main() {
   Sensor tempSensor("00:1A:2B:3C:4D:5E", 100);
   tempSensor.display();
}
Device configured with MAC: 00:1A:2B:3C:4D:5E
MAC: 00:1A:2B:3C:4D:5E, Rate: 100 Hz
```

3. Base Class Pointer Access and Object Slicing

Topic: Upcasting, static type checking and slicing

1) vPtr has the static type Vehicle*. C++ checks member access using the pointer's declared type, so only members declared in Vehicle can be called directly. The fact that the object at runtime is SportsCar does not make engageTurbo() a Vehicle member.

2) Vehicle plainVehicle = car copies only the Vehicle base portion. The SportsCar-specific turboEnabled member is not part of plainVehicle; this is object slicing.

3–4) Corrected code uses dynamic_cast for safe downcasting and a reference to avoid slicing:

```
#include <iostream>
class Vehicle {
public:
   int maxSpeed;
   Vehicle(int s) : maxSpeed(s) {}
   void honk() { std::cout << "General Horn\n"; }
   virtual ~Vehicle() = default;
};
class SportsCar : public Vehicle {
public:
   bool turboEnabled;
   SportsCar(int s, bool turbo) : Vehicle(s), turboEnabled(turbo) {}
   void engageTurbo() { std::cout << "Turbo Engaged!\n"; }
};
int main() {
   Vehicle* vPtr = new SportsCar(300, true);
   vPtr->honk();
   SportsCar* sportsPtr = dynamic_cast<SportsCar*>(vPtr);
   if (sportsPtr) sportsPtr->engageTurbo();
   SportsCar car(250, true);
   Vehicle& vehicleRef = car; // no slicing
   std::cout << vehicleRef.maxSpeed << '\n';
   delete vPtr;
}
```

Output:

```
General Horn
Turbo Engaged!
250
```

A virtual destructor is included so deleting a derived object through a base pointer is safe. A base reference or pointer preserves the original derived object; copying it into a base object by value causes slicing.

4. Incorrect Return Type in this Pointer Method Chaining

Topic: this pointer and fluent interface

1) setWidth returns WindowBox*. The expression box.setWidth(800) is therefore a pointer. The dot operator . requires an object, not a pointer; pointer member access uses ->. This makes the following .setHeight(600) invalid.

2) this is a pointer to the current object (WindowBox*). *this is the object itself as an lvalue (WindowBox& when returned by reference).

3–4) Corrected code:

```
#include <iostream>
class WindowBox {
   int width, height;
public:
   WindowBox() : width(0), height(0) {}
   WindowBox& setWidth(int width) {
      this->width = width;
      return *this;
   }
   WindowBox& setHeight(int height) {
      this->height = height;
      return *this;
   }
   void display() const {
      std::cout << "Window: " << width << " x " << height << '\n';
   }
};
int main() {
   WindowBox box;
   box.setWidth(800).setHeight(600);
   box.display();
}
```

Output:

```
Window: 800 x 600
```

5. Private Inheritance Access Violations and Inaccessible Base Cast

Topic: Access specifiers and upcasting

1) Under private inheritance, public and protected members of RawSocket become private members of SecureChannel. They can be used inside SecureChannel, but ordinary client code cannot call channel.closePort().

2) Upcasting from SecureChannel* to RawSocket* is inaccessible outside the derived class because RawSocket is a private base. Public upcasting is permitted when the inheritance relationship is publicly accessible at the point of conversion.

3) Two approaches: (A) expose the selected base member with using RawSocket::closePort; inside SecureChannel's public section; (B) use public inheritance if SecureChannel is an is-a RawSocket, or use composition and forward the required operations.

4) Corrected code using Approach A:

```
#include <iostream>
```

```
class RawSocket {
public:
   void openPort(int port) { std::cout << "Port " << port << " opened.\n"; }
   void closePort() { std::cout << "Port closed.\n"; }
};
class SecureChannel : private RawSocket {
public:
   using RawSocket::closePort;
   void establishEncryptedTunnel(int port) {
      openPort(port);
      std::cout << "Tunnel encrypted with TLS 1.3\n";
   }
};
int main() {
   SecureChannel channel;
   channel.establishEncryptedTunnel(443);
   channel.closePort();
}
```

```
Port 443 opened.
Tunnel encrypted with TLS 1.3
Port closed.
```

SECTION B – PSEUDO-CODE WRITING QUESTIONS

1. University ERP Hybrid Hierarchy with Virtual Base Classes

```
CLASS Person
  PRIVATE personID, fullName, email
  CONSTRUCTOR(id, name, mail)
    personID <- id; fullName <- name; email <- mail

CLASS Student VIRTUAL EXTENDS Person
  PRIVATE gpa, creditsEarned
  CONSTRUCTOR(id, name, mail, grade, credits)
    CONSTRUCTOR Person(id, name, mail)
    gpa <- grade; creditsEarned <- credits

CLASS Faculty VIRTUAL EXTENDS Person
  PRIVATE academicDepartment, rankTitle
  CONSTRUCTOR(id, name, mail, dept, rank)
    CONSTRUCTOR Person(id, name, mail)
    academicDepartment <- dept; rankTitle <- rank

CLASS TeachingAssistant EXTENDS Student, Faculty
  PRIVATE stipendAmount, assignedCourse
  CONSTRUCTOR(id, name, mail, grade, credits, dept, rank, stipend, course)
    CONSTRUCTOR Person(id, name, mail)  // most-derived initializes virtual base
    CONSTRUCTOR Student(id, name, mail, grade, credits)
    CONSTRUCTOR Faculty(id, name, mail, dept, rank)
    stipendAmount <- stipend; assignedCourse <- course
  METHOD DisplayTARecord()
    PRINT personID, fullName, gpa, academicDepartment, stipendAmount
END CLASS
```

Trace: Create TeachingAssistant(501, “Anu”, “anu@uni.edu”, 8.7, 90, “IT”, “Assistant Professor”, 12000, “C++”). One shared Person is initialized first, then Student, Faculty, and finally TeachingAssistant.

Illustrative output: ID: 501 | Name: Anu | GPA: 8.7 | Department: IT | Stipend: 12000

2. Polymorphic Payroll with Abstract Base Class and Base Pointers

```
ABSTRACT CLASS Employee
  PROTECTED employeeID, name
  CONSTRUCTOR(id, n) employeeID <- id; name <- n
  ABSTRACT METHOD CalculatePay() RETURNS REAL
```

```
  METHOD DisplayHeader() PRINT employeeID, name

CLASS SalariedEmployee EXTENDS Employee
  PRIVATE monthlySalary
  CONSTRUCTOR(id, n, salary) BASE(id,n); monthlySalary <- salary
  METHOD CalculatePay() RETURNS REAL
    RETURN monthlySalary

CLASS HourlyEmployee EXTENDS Employee
  PRIVATE hourlyWage, hoursWorked
  CONSTRUCTOR(id,n,wage,hours) BASE(id,n)
    hourlyWage <- wage; hoursWorked <- hours
  METHOD CalculatePay() RETURNS REAL
    IF hoursWorked <= 40 RETURN hourlyWage * hoursWorked
    ELSE RETURN 40*hourlyWage + (hoursWorked-40)*hourlyWage*1.5

MAIN
  DECLARE employees AS ARRAY OF POINTER TO Employee
  employees[0] <- NEW SalariedEmployee("E01","Ravi",8500)
  employees[1] <- NEW HourlyEmployee("E02","Maya",50,48)
  total <- 0
  FOR EACH employee IN employees
    employee->DisplayHeader()
    pay <- employee->CalculatePay()
    PRINT pay
    total <- total + pay
    DELETE employee
  PRINT total
END MAIN
```

Dry-run table:

Employee	Calculation	Pay	
Salaried	Fixed monthly salary	$8,500	
Hourly	40×50 + 8×50×1.5	$2,600	
Total	8,500 + 2,600	$11,100	

3. Composite Canvas Engine with Member Classes and this Chaining

```
CLASS Point2D
  PRIVATE x, y: REAL
  CONSTRUCTOR(x=0, y=0) this.x <- x; this.y <- y
  METHOD Set(xNew, yNew) x <- xNew; y <- yNew

CLASS Circle
  PRIVATE centerPoint: Point2D
          radius: REAL
  CONSTRUCTOR()
    centerPoint <- Point2D(0,0); radius <- 1.0
  METHOD SetCenter(x,y) RETURNS REF Circle
    centerPoint.Set(x,y)
    RETURN REF THIS
  METHOD SetRadius(r) RETURNS REF Circle
    IF r > 0 radius <- r
    ELSE PRINT "Invalid radius"
    RETURN REF THIS
  METHOD Draw()
    PRINT centerPoint coordinates and radius
END CLASS
```

Trace: SetCenter(12.5, 30.0) updates the contained Point2D. SetRadius(7.5) stores the valid radius. Each setter returns a reference to the same Circle, allowing the next method call.

Output: Circle center = (12.5, 30.0), radius = 7.5

4. Multi-Level Autonomous Fleet Telemetry with Base Pointers

```
CLASS Vehicle
  PROTECTED vehicleVIN, baseWeightKg
  CONSTRUCTOR(vin, weight)
    vehicleVIN <- vin; baseWeightKg <- weight
  METHOD DisplaySpecs()
    PRINT vehicleVIN, baseWeightKg

CLASS AutonomousVehicle EXTENDS Vehicle
  PROTECTED lidarSensorID, firmwareVersion
  CONSTRUCTOR(vin, weight, lidar, firmware)
    CONSTRUCTOR Vehicle(vin, weight)
    lidarSensorID <- lidar; firmwareVersion <- firmware

CLASS DeliveryDrone EXTENDS AutonomousVehicle
  PRIVATE payloadCapacityKg, batteryChargePercent
  CONSTRUCTOR(vin, weight, lidar, firmware, payload, battery)
    CONSTRUCTOR AutonomousVehicle(vin, weight, lidar, firmware)
    payloadCapacityKg <- payload; batteryChargePercent <- battery
  METHOD DisplayFlightStatus()
    CALL DisplaySpecs()
    PRINT lidarSensorID, firmwareVersion, payloadCapacityKg, batteryChargePercent

MAIN
  dronePtr <- NEW DeliveryDrone("VIN123", 12.5, "LDR9", "v2.1", 4.0, 85)
  basePtr AS POINTER TO Vehicle <- dronePtr
  basePtr->DisplaySpecs()
  derivedPtr AS POINTER TO DeliveryDrone <- DOWNCAST(dronePtr)
  derivedPtr->DisplayFlightStatus()
  DELETE dronePtr
END MAIN
```

Constructor order: Vehicle first, then AutonomousVehicle, then DeliveryDrone. Destruction occurs in reverse order. The base pointer can access Vehicle members; a derived pointer is needed for drone-specific telemetry.

5. Dynamic Object Pointer Array and Zero-Copy Sorting Engine

```
CLASS Product
  PUBLIC sku: STRING
         price: REAL
         stockCount: INTEGER
  CONSTRUCTOR(code, cost, stock)
    sku <- code; price <- cost; stockCount <- stock
  METHOD PrintProduct()
    PRINT sku, price, stockCount

MAIN
  catalog[0] <- NEW Product("Laptop",1200,10)
  catalog[1] <- NEW Product("Mouse",25,50)
  catalog[2] <- NEW Product("Monitor",350,15)
  FOR i <- 0 TO N-2
    FOR j <- 0 TO N-i-2
      IF catalog[j]->price > catalog[j+1]->price
        tempPtr <- catalog[j]
        catalog[j] <- catalog[j+1]
        catalog[j+1] <- tempPtr
      END IF
    END FOR
  END FOR
  FOR EACH ptr IN catalog
    ptr->PrintProduct()
  FOR EACH ptr IN catalog
    DELETE ptr
END MAIN
```

Dry-run pointer sorting:

Pass / comparison	Compared prices	Action	Pointer order after action
Initial	—	—	Laptop → Mouse → Monitor
1: Laptop vs Mouse	1200 > 25	Swap pointers	Mouse → Laptop → Monitor
1: Laptop vs Monitor	1200 > 350	Swap pointers	Mouse → Monitor → Laptop
2: Mouse vs Monitor	25 < 350	No swap	Mouse → Monitor → Laptop
Final	Ascending	Complete	Mouse → Monitor → Laptop

Sorted output order: Mouse ($25), Monitor ($350), Laptop ($1200). Only pointer variables are swapped; the Product objects remain at their original memory addresses. Finally, each dynamically allocated object is deleted exactly once.